

FRANKFURT UNIVERSITY OF APPLIED SCIENCES

Safety Critical Computer Systems

Assignment-2

Prepared for:

Prof. Dr. Mathias Wagner

Prepared by:

- 1. Hardikkumar Dudhat (Matriculation No: 1381838)**
- 2. Milan Dabhi (Matriculation No: 1381854)**
- 3. Milankumar Goti (Matriculation No: 1381867)**
- 4. Parth Desai (Matriculation No: 1381870)**

Question 1:

Emergence:

- Emergence as the characteristics of complex system is ubiquitous, and it has practical significance to make expected function emerged through the emergence control.
- As a science concept, emergence is used to describe the modes and structures or properties of complex systems, it is also an essential characteristic of complex systems.

Emergent Phenomena:

- In some publications, the occurrence of a deadlock in a computer system is called an emergent phenomenon. With the advent of multi-programming computer systems, the following event has been occasionally observed: when executing a number of processes concurrently, the system comes to a permanent halt, although each process, executed in isolation executes flawlessly.
- Artificial emotion as a phenomenon resulting from a series of modifications to emergent behaviors generated by a behavior-based artificial intelligence (AI) approach.

Question 2:

```
## Python code for Mandelbrot Fractal
```

```
# Import necessary libraries
```

```
from PIL import Image
```

```
from numpy import complex, array
```

```
import colorsys
```

```
# setting the width of the output image as 1024
```

```
WIDTH = 1024
```

```
# a function to return a tuple of colors
```

```
# as integer value of rgb
```

```
def rgb_conv(i):
```

```
    color = 255 * array(colorsys.hsv_to_rgb(i / 255.0, 1.0, 0.5))
```

```
    return tuple(color.astype(int))
```

```
# function defining a mandelbrot
```

```
def mandelbrot(x, y):
```

```
    c0 = complex(x, y)
```

```
    c = 0
```

```

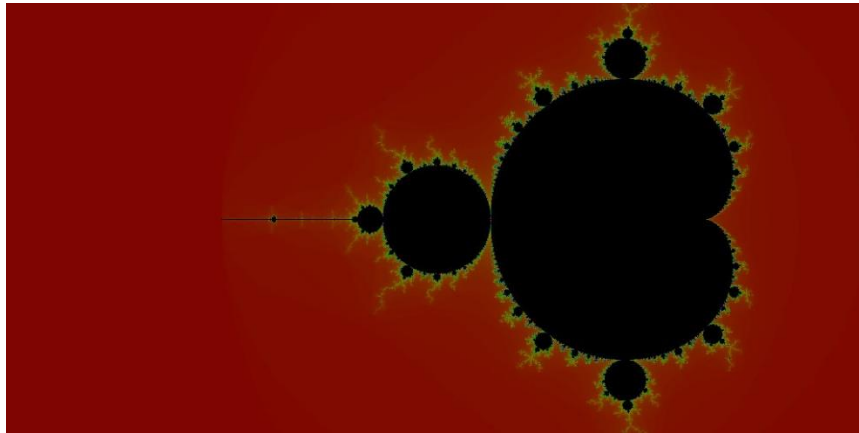
for i in range(1, 1000):
    if abs(c) > 2:
        return rgb_conv(i)
    c = c * c + c0
return (0, 0, 0)

# creating the new image in RGB mode
img = Image.new('RGB', (WIDTH, int(WIDTH / 2)))
pixels = img.load()

for x in range(img.size[0]):
    # displaying the progress as percentage
    print("%.2f %% " % (x / WIDTH * 100.0))
    for y in range(img.size[1]):
        pixels[x, y] = mandelbrot((x - (0.75 * WIDTH)) / (WIDTH / 4), (y -
            (WIDTH / 4)) / (WIDTH / 4))
# to display the created fractal after
# completing the given number of iterations
img.show()

```

Output:



Question 3:

A) Reviews of Literature:

Software measurement process must be a good oriented methodical process that measures, evaluates, adjusts, and finally improves the software development process (Shanthi and Duraiswamy, 2011). Software measurement has become a key aspect of good software engineering practice (Farooq and Quadri, 2011). Software metrics deals with the

measurement of software product and software product development process and it guides and evaluating models and tools.

Metrics are managements of different aspects of an endeavor that help us determine whether or not we are progressing toward the goal of that endeavor. Many software measures activities have been proposed in the literature, some of them are (Baumert and McWihinnet, 1992; Hammer et al., 1998; Janakiram and Rajasree, 2005; Loconsole, 2001; Paulk et al., 1993). Software metrics can be classified under different categories although same metrics may belong to more than category. Software metrics that are broken up into five categories (1) Commercial perspective (2) Significance perspective (3) Observation perspective (4) Measurement perspective (5) Software development perspective (Farooq and Quadri, 2011).

B) Categories of Metrics

There are 3 types of software metrics:

- **Product Metrics:**
Product metrics are used to evaluate the state of the product, tracing risks and uncovering prospective problem areas. The ability of team to control quality is evaluated.
- **Process Metrics:**
Process metrics pay particular attention on enhancing the long-term process of the team or organization.
- **Project Metrics:**
Project matrix describes the project characteristic and execution process such as number of software developer, staffing pattern over the life cycle of software, cost and schedule, and productivity.

C) Complexity Metrics

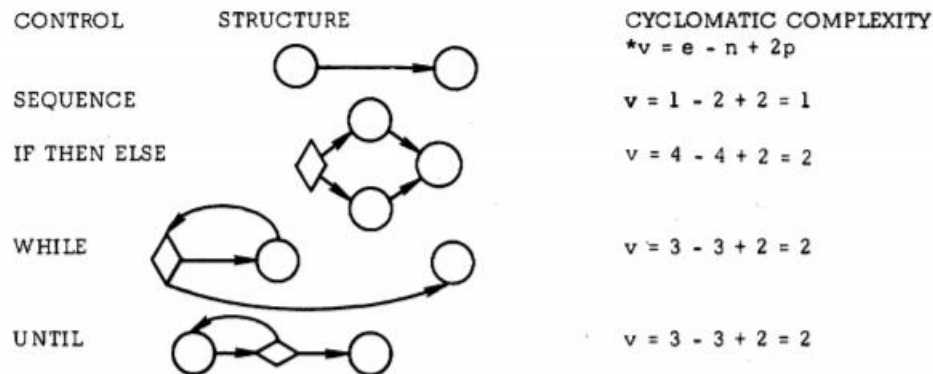
Metrics for measuring software complexity have many applications in software engineering activities including analysis, testing, debugging, and maintenance of programs, and management of project. However, although a number of complexity metrics have been proposed and studied for sequential and/or centralized software, until recently little attention was focused on complexity metrics for concurrent and/or distributed software.

D) Cyclomatic Complexity

Cyclomatic Complexity is used as the basis for a testing methodology. The procedure here is for the measuring the complexity of a program by computing the number of linearly

independent paths, control the "size" of program by setting an upper limit to paths, and use cyclomatic complexity as a basis for testing a program.

(E) How can the Cyclomatic Complexity be used?



Notice that the sequence of an arbitrary number of nodes always has unit complexity and that cyclomatic complexity conforms to our intuitive notion of "minimum number of paths".

Question – 4

A) Reliability

The reliability of a system is the probability of accomplishment of a function under specified environmental conditions and over a specified time. Reliability requirements are concerned with making a system failure-free.

Software Reliability is the probability of failure-free software operation for a specified period of time in a specified environment. Software Reliability is also an important factor affecting system reliability. It differs from hardware reliability in that it reflects the design perfection, rather than manufacturing perfection.

B)

$$P(F) = 4\%$$

$$P(F') = 96\%$$

$$P(D/F) = 90\%$$

$$P(D/F') = 2\%$$

Let $P(D)$ be the detection of Faulty.

$$P(D) = P(F) * P(D/F) + P(F') * P(D/F')$$

$$P(D) = (0.04) * (0.90) + (0.96) * (0.02)$$

$$P(D) = 0.0552$$

Now, $P(X)$ be the detection of Faulty which are actually Faulty.

$$P(X) = (P(F) * P(D/F)) / P(D) = \mathbf{0.6521}$$

The probability that a component which is recorded as faulty is actually faulty is 0.6521.

References:

1. Tianyun Huang, Xue-bo Chen, Wei Wang and Xin-yu Ouyang. Thinking in Emergence Control for Complex System. Taipei, Taiwan, 2011.
2. ZHAO Nan-nan, OUYANG Xin-yu. Summary on Emergence of Complex Systems. Qingdao, China, 2015.
3. Takashi Gomi and Joseph Ulvr. Artificial Emotions as Emergent Phenomena. Tokyo, Japan, 1993.
4. THOMAS J. MCCABE. A Complexity Measure. IEEE Transaction on software Engineering, VOL. SE-2, NO. 4, DECEMBER 1976.
5. Ming-Chang Lee, To Chang. Software Measurement and Software Metrics in Software Quality. Taiwan, Vol 7, No. 4, July 2013.
6. Amjan Shaik, CRK Reddy, A Damodaram. 2012. "Object Oriented Software Metric and Quality Assesment: Current State of the Art". International Journal of Computer Applications. Volume 37, Number 11, 2012.